

Parallelization Strategy

The application is parallelized using a hybrid MPI+CUDA approach, where the workload is distributed across nodes using MPI and accelerated locally using GPUs via CUDA. The dataset is partitioned across MPI processes by dividing the columns of the input matrix among the available ranks. Each MPI process is responsible for a subset of columns and performs computations independently on its assigned data.

The input matrix is read by the root process and distributed across all ranks using MPI_Scatterv. Column labels are partitioned using MPI_Scatterv, ensuring a balanced distribution even when the number of columns is not evenly divisible by the number of processes. The corresponding portions of the matrix is also distributed and stored in column major format to enable efficient memory access patterns on the GPU.

During execution, global synchronization is required when computing cluster statistics such as sums and counts. Each process computes partial results locally, which are then aggregated across all ranks using MPI_Allreduce. This ensures that all processes operate on consistent cluster averages.

The row labels are updated centrally on the root process based on the globally reduced distance values and then broadcasted to all other ranks using MPI_Bcast. This guarantees consistency of row labels across all processes before proceeding to the next iteration.

Work Division Across MPI Nodes

The dataset is partitioned across MPI process using a block distribution of columns. Each process receives approximately $\text{num_cols} / \text{size}$ columns, with any remaining columns assigned to the first few ranks to maintain load balance.

Each MPI process operates independently on its local subset, performing cluster sum calculations, partial row distance computations, and column label updates. Global communication is only introduced when required. Cluster statistics are aggregated using MPI_Allreduce, and updated row labels are synchronized across processes using MPI_Bcast.

This design ensures effective load balancing across nodes while limiting communication to essential synchronization points, thereby reducing overhead and improving scalability.

CUDA Parallelization Overview and Kernel Design

Within each MPI process, GPU acceleration is used to parallelize the main computational steps of the co-clustering algorithm, namely cluster average computation, row distance computation, and column label updates. Each of these steps is implemented as a separate CUDA kernel to exploit fine-grained parallelism and maximize GPU utilization. CUDA kernels execute asynchronously here, therefore explicit synchronization can be done further for precise timing measurements. However, the reported total execution time reflects the full GPU computation.

The cluster average computation is performed by the `calculate_local_sums_kernel`, where each thread is assigned to a column and iterates over all rows to accumulate partial cluster sums and counts. Since multiple threads may update the same cluster entries concurrently, atomic operations (`atomicAdd`) are used to ensure correctness. The row distance computation is implemented in the `compute_raw_dist_kernel`, where each thread processes a single row and computes its distance to all possible row clusters using the current column labels and cluster averages. This computation is independent for each row and therefore does not require synchronization or atomic operations. Finally, the column label update is handled by the `update_col_kernel`, where each thread processes one column and determines the closest cluster by evaluating distances across all rows. Atomic operations are used here to count the number of label changes across threads.

Overall, the kernels are designed with a one-dimensional thread mapping, where threads are assigned either to rows or columns depending on the operation. This approach ensures a balanced workload, sequential memory access patterns, and efficient utilization of GPU resources.

Thread Blocks and Grid Configuration

All CUDA kernels use a one-dimensional grid with a fixed block size of 256 threads, where the grid size is computed as $(N + 255) / 256$ to ensure full data coverage. Depending on the kernel, threads are mapped either to rows or columns. Row-based kernels assign one thread per row (`num_rows`), while column-based kernels assign one thread per local column (`local_cols`). This configuration provides a simple and efficient mapping of work to GPU threads while maintaining good occupancy and parallel execution.

Iterative Algorithm Flow

Each iteration consists of three main steps. First, cluster averages are computed through local GPU computations, followed by a global aggregation across MPI processes using `MPI_Allreduce`. Second, row labels are updated based on distances computed on the GPU, with results reduced across nodes and finalized on the root process before being broadcast to all ranks. Finally, column labels are updated entirely on the GPU, with each process independently updating its local subset. The algorithm terminates when no labels change or when the maximum number of iterations is reached.

Implementation	Dataset	Node	Time (s)	Speedup	Efficiency
Serial	Medium (m)	1	74.06		
MPI	Extra Large (xl)	1	763.19	1.0	100%
		2	1024	0.74	37%
		4	590	1.29	32%
		8	371	2.06	26%
CUDA	Extra -X Large (xxl)	1	439	1.00	100%
		2	236.78	1.85	92.5%
		4	127.76	3.43	85.8%
		8	74.71	5.88	73.5%

Table 1 Performance comparison of Serial, MPI, and CUDA+MPI implementations.

Performance Results

The correctness of the implementation was verified using the normalized mutual information (NMI) metric by comparing the generated labels with the reference solution. On the Small dataset, all implementations (serial, MPI, and CUDA+MPI) achieved an NMI of 1.0, confirming full correctness. For larger datasets, the CUDA+MPI implementation consistently achieved an NMI above 0.95, which satisfies the required correctness threshold. Overall, the application maintains high clustering quality while scaling to larger problem sizes, indicating that parallelization does not negatively impact solution accuracy.

MPI Performance

The MPI-only implementation shows limited scalability on the Extra Large (XL) dataset. Increasing the number of nodes from 1 to 2 initially leads to worse performance due to communication overhead outweighing computation. As the number of nodes increases further, execution time improves, achieving a speedup of 1.29 $\times$ on 4 nodes and 2.06 $\times$ on 8 nodes. However, efficiency drops significantly from 37% at 2 nodes to 26% at 8 nodes. This behavior is primarily caused by frequent global communication operations such as MPI_Allreduce and MPI_Bcast, which become a bottleneck as more nodes are added and the computation per node decreases.

CUDA+MPI Performance

In contrast, the CUDA+MPI implementation demonstrates strong scalability on the Extra Etra Large (XXL) dataset. GPU acceleration significantly improves the computation-to-communication ratio, leading to much better performance compared to MPI-only execution. A speedup of 1.85 $\times$ is achieved on 2 nodes, 3.43 $\times$ on 4 nodes, and 5.88 $\times$ on 8 nodes. The efficiency remains high, decreasing gradually from 92.5% at 2 nodes to 73.5% at 8 nodes. This indicates near-linear scaling and effective utilization of distributed GPUs. The improved performance is due to parallel execution of compute-intensive kernels on the GPU and reduced relative impact of MPI communication overhead. Overall, the CUDA+MPI implementation significantly outperforms the MPI-only version in both scalability and efficiency.

The results clearly show that GPU acceleration is essential for achieving scalable performance in co-clustering, as it increases computational intensity and mitigates communication bottlenecks present in the MPI-only implementation.

Scalability Graphs

1. MPI Speedup vs nodes (XL)

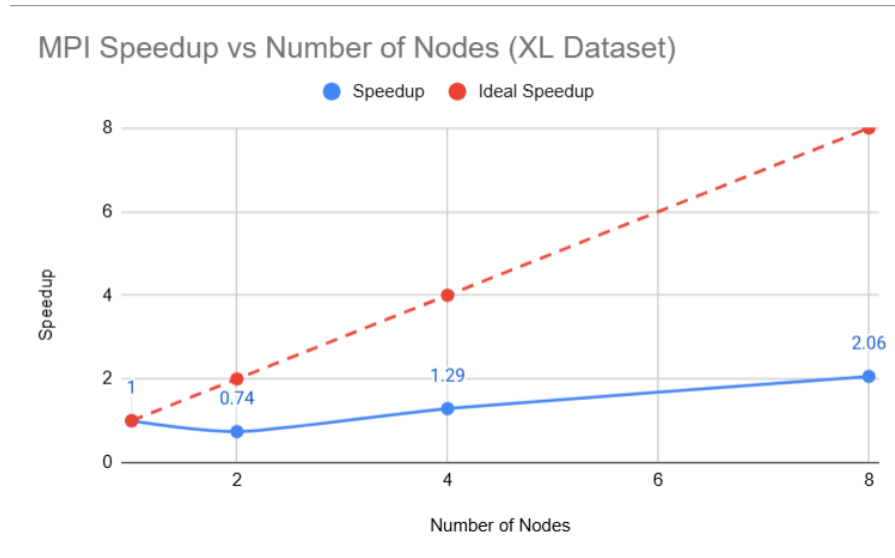


Figure 1 MPI speedup versus ideal scaling on the XL dataset.

2. MPI + CUDA speedup vs nodes (XXL)

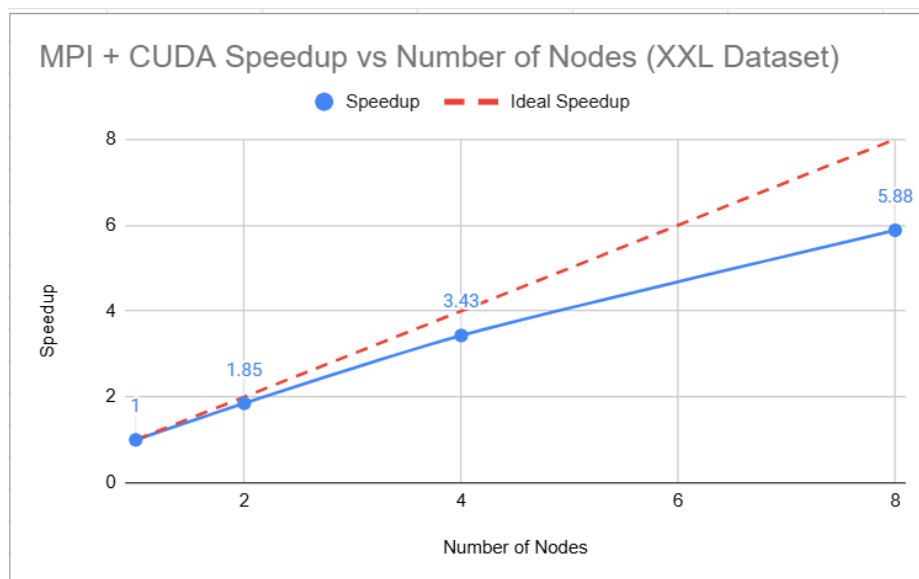


Figure 2 CUDA+MPI speedup versus ideal scaling on the XXL dataset.